LIFE CYCLE OF A SOFTWARE SYSTEM

Every software system, like any other engineered product, passes through a series of well-defined stages from the time it is conceived until it is retired. This sequence of stages is known as the Software Life Cycle or Software Development Life Cycle (SDLC). The life cycle of a software system provides a structured way to develop, operate, maintain, and eventually replace or retire software. **It ensures that the software meets the needs of users, is reliable, maintainable, and cost-effective.** Understanding this life cycle helps software engineers to plan and control projects effectively, minimize risks, and produce high-quality software systems. **The life cycle of a software system refers to the entire lifespan of a software product, beginning from its initial conception (idea or feasibility study) through development, deployment, maintenance, and finally to its decommissioning or replacement**. It defines how software evolves from birth to death, involving all the activities and processes that ensure its continuous usefulness.

The life cycle of a software system represents the structured journey of software from conception to retirement. It ensures that software is developed in a disciplined, organized, and cost-effective manner. By following a well-defined life cycle, organizations can ensure that their systems remain reliable, maintainable, and aligned with evolving user needs. In essence, the software life cycle is the heartbeat of software engineering—it gives order, predictability, and purpose to the complex process of building and maintaining software systems.

Objectives of a Software Life Cycle

- To provide a clear structure for software development and management.
- To ensure quality and reliability of the software system.
- To enable effective project planning and tracking.
- To control costs and schedule through defined processes.
- To make maintenance and upgrades manageable.
- To provide a systematic approach to retiring or replacing obsolete systems.

Major Phases in the Life Cycle of a Software System

The life cycle of a software system typically includes the following phases:

1. Feasibility Study and Problem Definition: This is the initial phase where the problem is identified, and possible solutions are analyzed. The goal is to determine whether the project is technically, economically, and operationally feasible.

- Understanding the client’s needs or business goals.
- Evaluating whether software can solve the problem.
- Estimating costs, timelines, and resource requirements.
- Preparing a feasibility report for management approval.
- To produce feasibility Report or Project Proposal.

2. Requirements Analysis and Specification

Once feasibility is established, the next step is to define what the system must do. This involves gathering, analyzing, and documenting user and system requirements.

- Conducting interviews, surveys, and workshops with stakeholders.
- Writing Software Requirements Specification (SRS) documents.
- Defining both functional and non-functional requirements.
- SRS document should be produced, this serves as a formal agreement between stakeholders and developers.

3. System Design

It involves both high-level design (architecture) and detailed design (components, interfaces, data structures).

- Designing system architecture (modules, layers, and components).
- Defining data models and database structures.
- Designing user interfaces and interaction flow.
- Producing Design Specification Documents.
- System and Software Design Documents will be produced.

4. Implementation (Coding)

Here, the design is translated into a working system using programming languages and tools. Each module is coded, tested, and integrated.

- Writing source code according to design specifications.
- Performing unit testing on individual components.
- Using version control systems to manage code changes.
- Executable software modules is the output

5. Integration and Testing

Individual modules are integrated into a complete system, and rigorous testing is done to ensure that the system works as expected. Types of testing include:

- Integration testing: Ensures that modules interact correctly.
- System testing: Validates the system as a whole.
- Acceptance testing: Confirms that requirements have been met.
- Output: Test reports and a verified software system ready for deployment.

6. Deployment (Installation and Delivery)

After successful testing, the system is delivered and deployed to the user environment.

- Installing software on production servers.
- Configuring the system to fit the user’s environment.
- Providing user manuals and training.

- Conducting final acceptance with users.
- Operational software system is produced

7. Operation and Maintenance

This is the longest phase of the software life cycle. Once the system is in use, it requires updates, fixes, and performance improvements.

Types of maintenance:

- Corrective Maintenance: Fixing bugs and errors.
- Adaptive Maintenance: Adjusting to environmental or business changes.
- Perfective Maintenance: Improving system performance or adding features.
- Preventive Maintenance: Preventing future problems through optimization.
- Updated versions, patches, and maintenance logs.

8. Retirement (Disposal or Replacement)
Eventually, the software becomes out-dated or no longer meets business needs.
It is then retired or replaced by a new system.

- Data migration to new systems.
- Archiving old records and documentation.
- Notifying users and decommissioning servers.
- Conducting post-mortem evaluation.
- System closure report and data handover.

Importance of Software Life Cycle

- Improved project control and predictability.
- Enhanced product quality through planned validation at every stage.
- Reduced risk of failure by identifying problems early.
- Better cost estimation and resource management.
- Ease of maintenance and scalability.
- Improved collaboration among stakeholders, developers, and users.